Glitch-Free Clock Multiplexer with Active Decision Logic, Symmetric Failover and Safe Recovery for Multi-Clock SoC Systems

Hai Minh Truong*, Phuong Vu Thi*, Thuan Van Le†
†Faculty of Electrical and Electronic Engineering, Phenikaa School of Engineering, Phenikaa University

***Abstract*—Dynamic and reliable switching among asynchronous clock sources is a persistent hurdle in advanced SoC and FPGA systems, where conventional clock multiplexers frequently suffer from glitches, metastability, or deadlock, especially when a clock source unexpectedly halts. This work introduces a glitch-free clock multiplexer architecture featuring autonomous and symmetric failover, permission-driven selection logic, and integrated activity sensing across clock domains. By combining cross-domain activity detection, a permission-based arbitration scheme, and a two-stage synchronizer with an augmented edge-capture mechanism, the design ensures clean, deterministic handover and robust system recovery without requiring an external reference clock. Asynchronous force-disable paths further guarantee safe transitions under all failure scenarios. Comprehensive simulation and FPGA prototyping on Intel Cyclone V validate the architecture's effectiveness across normal operations, clock failures, and recovery events, with resource consumption as low as 12 ALMs and up to 496.52 MHz operating frequency. Compared to timer-based and cross-coupled designs, the proposed solution demonstrably enhances resiliency and determinism while maintaining compact hardware footprint.**

***Index Terms*—Clock multiplexer, glitch-free switching, asynchronous clocks, failover, deadlock-free, FPGA, clock domain crossing**

I. Introduction

As digital systems grow in complexity and demand greater adaptability, reliable clock switching becomes a critical concern, especially in applications such as system-on-chip (SoC) platforms and FPGA-based designs [1]. Clock multiplexers, used to select between multiple clock sources for dynamic frequency scaling, power optimization, or fault recovery [2], must ensure robust and consistent operation to preserve system integrity [3].

A basic clock multiplexer can be implemented with simple combinational logic, as shown in the top of Fig. 1 [4]. The selection is governed by a digital control signal; for example, when the selector is low (`sel` $= 0$), the output `clk_out` follows the first clock input (`clk1`), and when the selector is high (`sel` $= 1$), it switches to the second input (`clk2`). While this approach is straightforward and area-efficient, its direct and unsynchronized nature gives rise to significant reliability hazards when switching between asynchronous clock domains.

A. Problem Definition

The conventional multiplexer design is prone to several critical failure modes. First is the generation of glitches, which are unintended, short-lived pulses at the output. This phenomenon typically occurs when the `sel` signal changes while the input clocks are active, creating a race condition in the logic. The circled region in Fig. 1 illustrates such a spurious spike [5], which can easily violate the timing requirements of downstream logic. Second, metastability can occur when the asynchronous `sel` signal is sampled too close to a clock edge, causing an indeterminate state that can propagate through the system. While this risk is commonly mitigated by passing the control signal through a two-stage flip-flop synchronizer, eliminating it entirely remains a challenge in high-speed designs. Finally, deadlock represents a catastrophic failure where the output becomes permanently stuck at a fixed logic level. As shown by the waveform in Fig. 2, this can happen if a selected clock halts in a high state, preventing the multiplexer from switching to an alternate clock and rendering the system unresponsive.

B. Related Work

Prior architectures have been developed to address these issues. The BBM technique [6], often implemented using cross-coupled flip-flops, was introduced to address the fundamental shortcomings of simpler combinational approaches, which are vulnerable to both metastability and glitches during clock transitions. By ensuring that only one clock enable is ever active during handover, BBM architectures effectively eliminate the risk of metastability and thus provide a substantial improvement in operational robustness [7]. However, despite its simplicity and effectiveness in resolving metastability, BBM does not fully address the glitch problem in all failure scenarios. In particular, if the selected clock halts while remaining in a high state, the output `clk_out` can become locked at logic high, failing to switch to a viable clock source and resulting in a deadlock condition. This vulnerability is illustrated in the lower part of Fig. 2, where the output remains stuck until the faulted clock resumes toggling. Thus, while BBM markedly advances reliability compared to earlier designs, it remains susceptible to output glitches and deadlock in the presence of certain clock failures, motivating the need for an even more robust solution.

Timer-based (watchdog) clock multiplexers [8] represent a significant improvement over BBM solutions through their use of dedicated timers to detect stalled clock domains, as illustrated in Fig. 3. When inactivity is detected, these circuits force the output to a defined safe state (typically

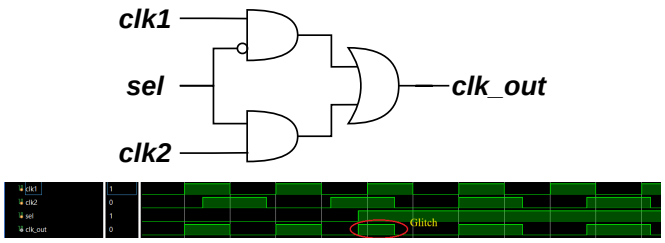

Fig. 1. Traditional AND/OR-based clock multiplexer: logic structure (top); waveform demonstrating output glitch caused by select changes during active clocks (bottom).

logic Low) by asserting a reset, effectively eliminating the risk of deadlock when a clock halts high. In particular, if the selected clock resumes toggling after a fault, the timer-based approach can automatically recover the output operation that `clk_out` will follow the recovered clock without further user intervention. However, the principal limitation of timer-based designs lies in their passive handover policy. If the selected clock fails and the selector signal (`sel`) remains unchanged, the output is merely forced to the safe state and does not transfer to an alternate live clock source. Only by either toggling the selector or waiting for the original clock to recover does normal operation resume. This lack of autonomous, immediate switchover to any available healthy clock restricts the system's applicability for high-availability or fail-operational scenarios, where uninterrupted output is essential. Furthermore, timer-based implementations incur a performance penalty, as their frequency of operation (maximum Fmax) is typically limited by the additional logic and constraints introduced by the watchdog circuits.

C. Motivation for This Work

The prior architectures each tackle select failure modes: combinational designs offer simplicity, BBM methods suppress glitches, and timer-based schemes add forced-safe outputs with some recovery. However, none alone achieves glitch immunity, deadlock freedom, and fully autonomous, immediate switchover to any healthy clock source.

Motivated by these gaps, our work aims to create a holistic solution with two primary objectives. The first is to implement fully autonomous failover, allowing the multiplexer to intelligently switch to a live clock source upon failure without any external intervention. The second is to enforce a safe-state condition, forcing the output to a stable logic low during the rare but critical event that both input clocks become inactive. Our architecture intentionally integrates the advantages of prior designs while overcoming their limitations, enabling robust failure detection, rapid switchover, and high-frequency operation within a single, unified framework.

D. Paper Organization

The remainder of this paper is organized as follows. Section II presents the detailed three-layered design of the proposed clock multiplexer. Section III provides simulation results to verify its functional correctness and presents a comparative analysis of its FPGA implementation performance. Finally, Section IV concludes the paper and suggests directions for future work.

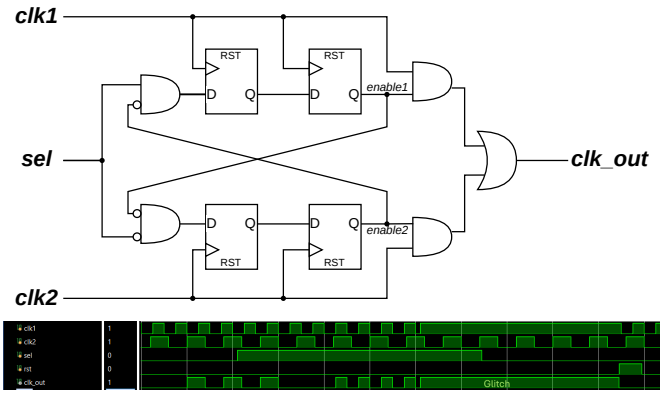

Fig. 2. Break-before-make multiplexer: cross-coupled flip-flop schematic (top); deadlock when a selected clock halts while high (bottom).

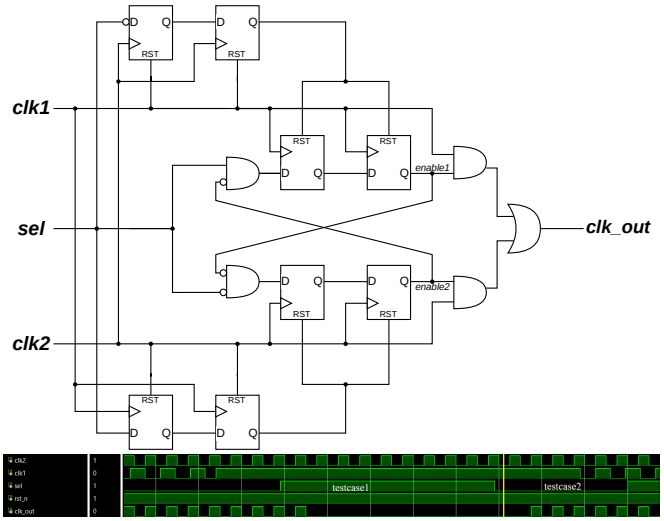

Fig. 3. Timer-based multiplexer: schematic with timers per clock source (top); waveform showing recovery upon selector change after clock failure (bottom).

II. Proposed Architecture

This section presents the detailed architecture of the proposed GFCM. The system is composed of three functionally distinct but closely interconnected layers, as illustrated in Fig. 4. This layered approach enables rapid failover and reliable, glitch-free operation during clock transitions. A key aspect of the design is its inherent safety during catastrophic failure scenarios, including the simultaneous loss of both input clocks. In a staggered dual-clock failure, the Activity Sensor asserts a `fail` signal upon detecting the first stopped

clock. This `fail` signal propagates to the Safe Handover layer, forcing `clk_out` to a safe logic-low state even if the second clock subsequently fails. However, in the rare simultaneous failure case, where both clocks stop within the same detection window, neither counter can detect activity from the other domain, so no `fail` signal is asserted. Consequently, `clk_out` safely holds its last valid state. This hold-state remains stable and deadlock-free until activity is detected from at least one clock, at which point the architecture autonomously recovers. These behaviors are illustrated in Figs. 7 and 6.

A. System Overview

Figure 4 displays the high-level block diagram of the proposed clock multiplexer. The circuit accepts two asynchronous clock inputs, `clk1` and `clk2`, and a user selector, `sel`, to produce a single, glitch-free output, `clk_out`. Internally, the design is decomposed into three sequential processing layers, each with clearly delineated responsibilities.

The first layer, the Activity Sensor, continuously monitors the health of each input clock by sampling it within the other clock's domain, generating real-time status signals that indicate if a clock has failed. These status signals are passed to the second layer, the Decision Making logic, which combines the health information with the user's preference to dynamically select the most appropriate clock source. This layer generates "desired clock" signals that represent the system's intent but do not directly control the output. Finally, the third layer, Safe Handover, receives this intent and converts it into the final output enables. It implements synchronized enable registers and mutual exclusion logic to ensure that switching is glitch-free and honors the break-before-make principle during all transitions.

This modular decomposition, as visually depicted in Fig. 4, clarifies both the dataflow and the control dependencies of the system, and forms the basis for ease of analysis and future extensibility.

B. Layer 1: Activity Sensor

The Activity Sensor layer interfaces directly with the two asynchronous clock domains. Utilizing cross-domain sampling, each clock is sampled in the other's domain, and transitions are detected by XORing consecutive samples [8]. A saturating synchronous counter, running in the opposing clock domain, accumulates the number of cycles since the last detected transition. If this counter exceeds a predefined threshold (set here to three cycles), a failure flag (`fail1` or `fail2`) is asserted for that clock. This approach robustly distinguishes between brief jitter and a true clock failure. It is important to note, however, that this reciprocal monitoring scheme has an inherent limitation: if both clocks fail simultaneously, both counters will halt, and neither failure flag can be asserted. The outputs of this layer, `fail1` and `fail2`, therefore represent the health status of `clk1` and `clk2` as observable from the other active domain.

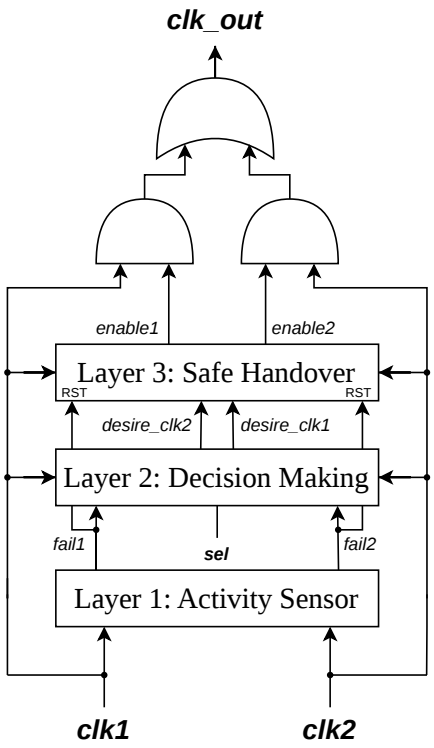

Fig. 4. Block diagram of the proposed clock multiplexer, illustrating the clear separation of monitoring, decision, and handover logic across three layers.

C. Layer 2: Decision Making

The Decision Making layer is responsible for the real-time assessment and selection of the appropriate clock source. Its operation is governed by a clear policy that uses the health status from Layer 1 (`fail1`, `fail2`) and the user's selection preference (`sel`) to determine the system's intent.

The policy prioritizes the clock chosen by the user, as long as it is functioning correctly. If the preferred clock fails while the alternate clock remains active, the logic immediately overrides the user's choice and designates the healthy clock as the desired source, enabling autonomous failover. Furthermore, if a failure is detected for both clocks (as in a staggered failure), this layer de-asserts both "desired clock" signals. This action is critical for driving the output to a safe, quiescent state, a process finalized by Layer 3. It is important to note that this layer's ability to command a safe state is entirely dependent on receiving at least one 'fail' signal from Layer 1. In the rare case of a perfectly simultaneous failure where no 'fail' signals are generated, this layer will be unaware of the fault and will not change its output. Throughout all operational modes, Layer 2 continually produces the "desired clock" signals that guide the final handover logic, ensuring a robust and rapid response to changing system conditions.

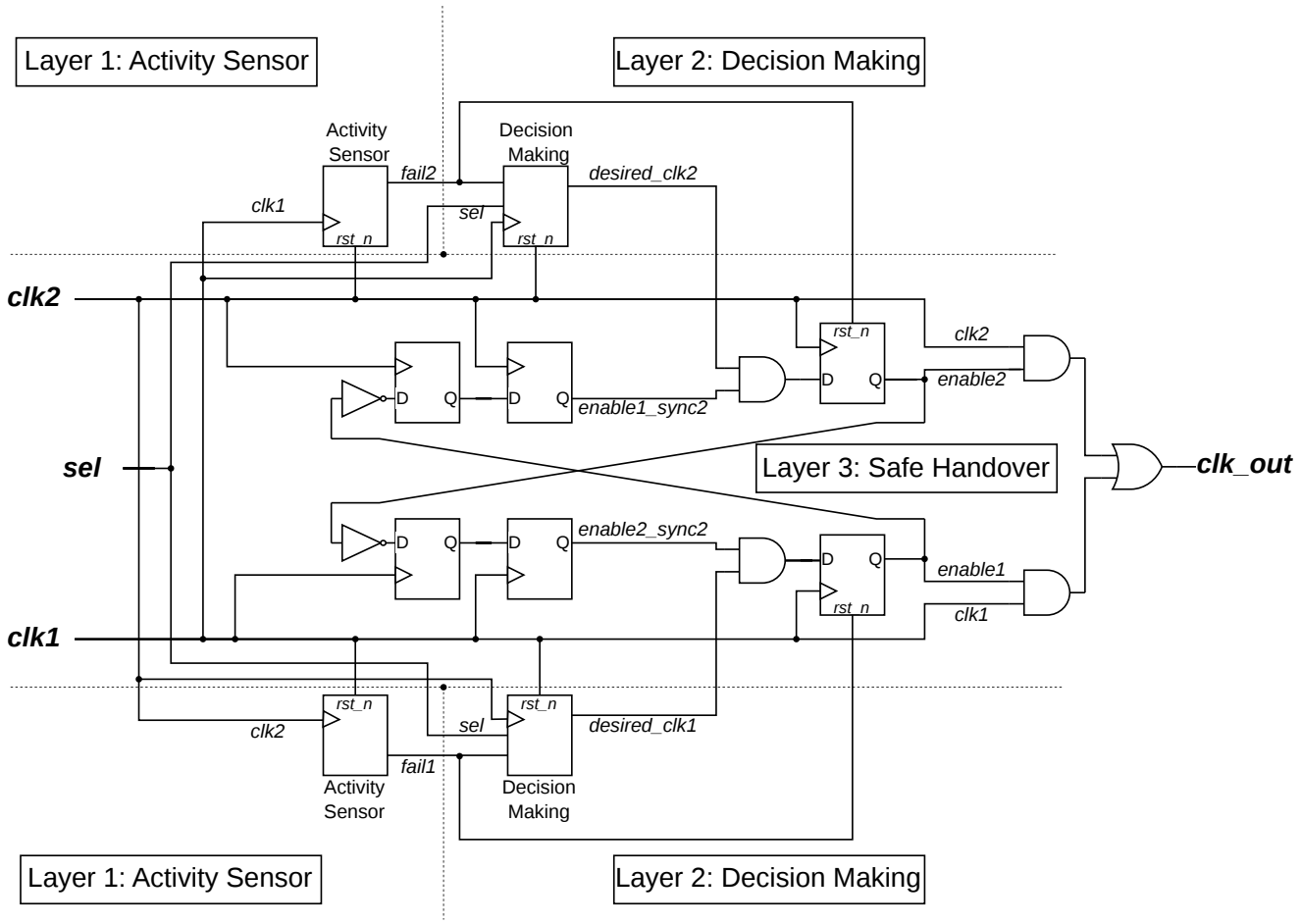

Fig. 5. Schematic of layered clock failover system, showing Activity Sensor, Decision Making, and Safe Handover logic for each clock domain. Mutual exclusion, synchronization, and safe output enforcement are all handled in Layer 3.

D. Layer 3: Safe Handover

The final layer, Safe Handover, is responsible for safely implementing the clock selection determined by the Decision Making layer. Its core objective is to ensure glitch-free, mutually exclusive switching between the two clock sources, guaranteeing that only one source is ever enabled at a time and that transitions do not cause spurious output or timing hazards. This is accomplished through several key mechanisms, whose high-level arrangement is shown in Fig. 4 and whose detailed implementation is provided in Fig. 5.

First, the ”desired clock” signals from Layer 2 are passed through dedicated flip-flop stages within their respective clock domains. These synchronizers are crucial for aligning the enable signals safely to each clock, pipelining the logic for improved timing, and ensuring enables only change on stable clock edges. This approach prevents the hazards common to direct combinational gating.

Second, mutual exclusion logic ensures that both output enables can never be active simultaneously. This enforces a strict break-before-make switching protocol, which produces clean, monotonic transitions at the output without the risk of contention.

Finally, the output gating logic provides a safe output state by holding `clk_out` low whenever no valid clock is available, such as when both inputs are inactive or a reset is asserted. By integrating enable synchronization, mutual exclusion, and safe-state handling, this layer provides a robust framework for all clock switching operations.

In summary, the architecture described above establishes a robust and flexible framework for safe clock switching, combining enable synchronization, mutual exclusion, and well-defined safe-state handling. The high-level structure of the system is captured in the block diagram, as shown in Fig. 4, which illustrates the interplay between clock activity sensing, decision making, and output gating.

To provide deeper insight into the practical realization of these mechanisms, the complete logic is detailed in the accompanying schematic diagram, as depicted in Fig. 5. This schematic reveals the precise signal paths, flip-flop stages, and gating elements that implement each architectural

function, clarifying how abstract principles are mapped to concrete hardware. Readers are encouraged to refer to this diagram for a comprehensive understanding of the internal circuitry and to see how each conceptual block is instantiated in the final design.

III. Results and Evaluation

A. Simulation and Functional Verification

To validate the correctness and reliability of the proposed design, a comprehensive functional verification was performed. The architecture was modeled in Verilog, and a dedicated testbench was developed to generate the stimulus for a range of critical scenarios. This testbench instantiates the proposed clock multiplexer as the Design Under Test (DUT) and uses procedural blocks to precisely control the behavior of the input clocks, selector, and reset signals. The resulting waveforms confirm the architecture's robustness in both failover and recovery scenarios.

The simulation waveform in Fig. 6 demonstrates the system's nuanced handling of multiple, complex failure and recovery scenarios. The first circled region highlights the architectural corner case where `clk1` fails, followed by `clk2` failing shortly thereafter. Because the time between the two failures is less than the detection threshold, the system does not assert a 'fail' signal and the output (`clk_out`) correctly holds its last valid state. The second circled region shows the system's primary failover mechanism: when the selected clock (`clk1`) fails while the alternate (`clk2`) actives, the output seamlessly switches to track `clk2`. Finally, the third circled region demonstrates automatic recovery, where the output returns to tracking the originally selected clock (`clk1`) as soon as it resumes toggling after its period of inactivity. This single waveform validates the design's ability to handle autonomous switching, hold-state on near-simultaneous failure, and full recovery, confirming its comprehensive robustness.

Figure 7 presents the critical test case of a staggered dual-clock failure. Here, the selected clock (`clk2`) fails first. Critically, there is a sufficient time delay before the alternate clock (`clk1`) also fails, allowing the Activity Sensor in the `clk1` domain to detect the failure of `clk2`. This triggers a `fail2` signal, which commands the Decision Making layer to de-assert all "desired clock" signals. As shown in the circled region, this action correctly forces the output `clk_out` to a safe, logic-low state before the entire system halts. This result validates the design's ability to handle staggered dual-failures gracefully, a scenario where many conventional designs would fail unpredictably. This behavior is distinct from the theoretical case of a perfectly simultaneous failure, where the output would hold its last state as described in Section II.

Collectively, these simulations validate the complete operational spectrum of the proposed architecture. The results confirm its ability to not only handle standard failover and recovery, but also to behave deterministically and safely in both staggered and near-simultaneous dual-failure corner cases. This proven robustness in complex fault scenarios demonstrates a clear advantage in safety and reliability over conventional clock-switching designs.

Fig. 6. Simulation demonstrating the system's key autonomous behaviors, showing the output holding its last state during a near-simultaneous failure (first circle), performing a seamless failover to the alternate clock (second circle), and automatically recovering to the primary clock upon its return (third circle).

Fig. 7. Simulation of a staggered dual-clock failure. The system detects the failure of the first clock and, as shown in the circle, safely forces the output to logic low before the second clock also fails.

B. FPGA Implementation and Resource Utilization

To evaluate performance and cost, all architectures were designed in Verilog HDL and synthesized for an Intel Cyclone V 5CEFA2F23C8 FPGA. The measured resource utilization, including Adaptive Logic Modules (ALMs), Logic Array Blocks (LABs), Adaptive Look-Up Tables (ALUTs), and Dedicated Logic Registers (DLRs)[1], is summarized in Table I. This allows for a direct comparison under consistent tool and device conditions.

The results in Table I reveal a clear progression in clock multiplexer design, highlighting the trade-offs between area, performance, and functional safety. The conventional BBM architecture [6] is the most area-efficient (3 ALMs) but provides no safety guarantees beyond being glitch-free. The timer-based approach [8] adds deadlock-free capability but does so at the cost of increased area (12 ALMs) and a lower maximum frequency.

The clock-reference-based design [9] achieves a significant milestone by providing full dual-stop safety. However, this robustness is dependent on the availability of a third, external, and continuously reliable reference clock, which adds system-level complexity. This design also has a considerable resource footprint of 27 ALMs.

In contrast, our proposed GFCM provides the same complete set of safety features as the clock-reference-based design, including dual-stop safety, but crucially, it operates without requiring an external reference clock. As shown in Table I, the GFCM achieves this high level of safety using only 14 ALMs, which is nearly half the resources of the

[1]These terms refer to key FPGA resources: ALUTs are the basic combinational logic elements; DLRs are the flip-flops; ALMs are the fundamental building blocks containing both ALUTs and DLRs; and LABs are clusters of multiple ALMs.

Table I: Feature and Implementation Comparison for Various Clock Multiplexer Designs.

Design	**Glitch-Free**	**Ref. Clock**	**Deadlock-Free**	**Dual-Stop Safety**	**Latency Bound**	**ALMs**	**LABs**	**ALUTs**	**DLRs**	**Max Freq. (MHz)**
Break-before-make [5], [6]	Yes	No	No	No	Constant	3	2	4	6	580.05
Timer-based [8]	Yes	No	Yes	No	Freq.-dep.	12	4	5	10	367.92
Clock-ref-based [9]	Yes	Yes	Yes	Yes	Constant	27	4	51	41	352.61
Proposed GFCM	Yes	No	Yes	Partial*	Constant	14	3	24	22	515.46

* The proposed GFCM provides full dual-stop safety (forces `clk_out` to logic low) for staggered failures via the `fail` signal. In the rare simultaneous failure case (both clocks stop within the same detection window), no `fail` signal is asserted and `clk_out` safely holds its last valid state until at least one clock recovers. See Section II for detailed explanation.

clock-reference-based design. Furthermore, at 515.46 MHz, its maximum frequency is substantially higher. The proposed GFCM offers a highly favorable trade-off among safety, performance, and hardware cost, as it delivers the highest level of functional safety and high performance while reducing resource consumption and avoiding system-level dependence on a third reference clock.

IV. Conclusion

This paper has presented a robust, fully autonomous GFCM architecture designed for high-reliability systems. The proposed design advances the state-of-the-art by providing a complete safety solution that ensures glitch-free switching, deadlock-free operation, automatic recovery, and critically, safe handling of staggered dual-stop events and deterministic hold-state behavior under perfectly simultaneous clock loss, all without the need for an external reference clock.

Functional verification through simulation confirmed the architecture's correct behavior across a range of complex fault scenarios, including both staggered and near-simultaneous clock failures. The proposed GFCM achieves the highest level of functional safety while operating at a maximum frequency of 515.46 MHz which is significantly faster than other designs that offer a subset of these safety features. This demonstrates a superior trade-off, delivering maximum reliability and high performance with only a modest resource cost, and eliminating the system-level dependency on a third reference clock required by other safe designs. Future work will focus on two primary directions. First, the architecture can be extended to support multiplexing of more than two clock domains, addressing the needs of more complex SoC. Second, further optimizations will be explored for ultra-low-power applications by investigating adaptive thresholding for failure detection and implementing resource-minimized monitoring logic.

References

[1] T. Xanthopoulos, *Clocking in Modern VLSI Systems*, ser. Integrated Circuits and Systems. Springer US, 2009.

[2] E. Kyriakakis, K. Tange, N. Reusch, E. O. Zaballa, X. Fafoutis, M. Schoeberl, and N. Dragoni, "Fault-tolerant clock synchronization using precise time protocol multi-domain aggregation," in *2021 IEEE 24th International Symposium on Real-Time Distributed Computing (ISORC)*, 2021, pp. 114–122.

[3] K. Vipin and S. A. Fahmy, "Fpga dynamic and partial reconfiguration: A survey of architectures, methods, and applications," *ACM Computing Surveys (CSUR)*, vol. 51, no. 4, pp. 1–39, 2018.

[4] J. Wakerly, *Digital Design Principles And Practices 4Th Ed.* Prentice-Hall Of India Pvt. Limited, 2006.

[5] R. Ginosar, "Fourteen ways to fool your synchronizer," in *Ninth International Symposium on Asynchronous Circuits and Systems, 2003. Proceedings.*, 2003, pp. 89–96.

[6] D. Harris and S. Harris, *Digital Design and Computer Architecture*. Morgan Kaufmann, 2012.

[7] K. R. Talupuru and S. Athi, "Achieving glitch-free clock domain crossing signals using formal verification, static timing analysis, and sequential equivalence checking," in *2011 12th International Workshop on Microprocessor Test and Verification*, 2011, pp. 5–9.

[8] S. Zeidler, O. Schrape, A. Breitenreiter, and M. Krstić, "A glitch-free clock multiplexer for non-continuously running clocks," in *2020 23rd Euromicro Conference on Digital System Design (DSD)*, 2020, pp. 11–15.

[9] T. V. Le and N. Van Dinh, "Glitch-free clock multiplexer using reference-clock missing-pulse detection and safe-off recovery for SoC applications," in *2025 International Conference on Engineering and Emerging Technologies (ICEET)*, 2025, pp. 1–6.